МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторная работа №5
по курсу «Программирование графических процессоров»

Сортировка чисел на GPU. Свертка, сканирование, гистограмма.

Выполнил: Немкова А. Р.
Группа: М8О-408Б-22
Преподаватели: А.Ю. Морозов,
Е.Е. Заяц

Москва, 2025

Условие

Цель работы. Ознакомление с фундаментальными алгоритмами GPU: свертка (reduce), сканирование (blelloch scan) и гистограмма (histogram). Реализация одной из сортировок на CUDA. Использование разделяемой и других видов памяти. Исследование производительности программы с помощью утилиты nvprof.

Входные данные. В первых четырех байтах записывается целое число n - длина массива чисел, далее следуют n чисел типа, заданного вариантом.

Выходные данные. В бинарном виде записывают n отсортированных по возрастанию чисел.

Вариант 2. Сортировка подсчетом. Диапазон от 0 до $2^{24} - 1$

Требуется реализовать сортировку подсчетом для чисел типа int. Должны быть реализованы:

- Алгоритм гистограммы, с использованием атомарных операций.
- Алгоритм сканирования для любого размера, с рекурсией и бесконфликтным использованием разделяемой памяти.

Ограничения: $n \leq 135 * 10^6$

Программное и аппаратное обеспечение

Графический процессор:

- Модель: NVIDIA GeForce RTX 3050 Laptop GPU
- Compute Capability: 8.6
- Объем графической памяти: 4 294 443 008
- Разделяемая память на блок: 49 152
- Количество регистров на блок: 65 536
- Размер warp: 32
- Максимальное количество потоков на блок: (1024 × 1024 × 64)
- Максимальное количество блоков: (2 147 483 647 × 65 535 × 65 535)
- Объем константной памяти: 65 536
- Количество мультипроцессоров (SM): 16

Центральный процессор:

- Модель: AMD Ryzen 7 5800H with Radeon Graphics 3.20 GHz
- Количество ядер: 8
- Количество потоков: 16
- Базовая тактовая частота: 3,2 ГГц
- Максимальная тактовая частота: 4,4 ГГц
- Кеш L1: 64 Кб (на ядро)
- Кеш-память L2: 512 КБ (на ядро)
- Кеш L3: 16 Мб

Оперативная память:

- Тип памяти: DDR4
- Объём памяти: 2 * 8 Гб
- Частота: 3200 Мт/с

Жёсткий диск:

- Тип: SSD
- Модель: SK Hynix HFM001TD3JX013N

- Объём памяти: 1 Тб
- Скорость записи/чтения: 3100/3500 МБ/с

Программное обеспечение:
- OS: Ubuntu 20.04 Linux (WSL2)
- IDE: VS Code
- Compiler: nvcc, gcc

Метод решения

Для решения задачи сортировки массива подсчетом на GPU использовались основные этапы: гистограмма, сканирование и сортировка подсчетом.

Первым шагом является построение гистограммы, которая отображает количество вхождений каждого значения в массиве. Гистограмма создается с использованием атомарных операций, что позволяет многим потокам параллельно обновлять количество вхождений для каждого значения без конфликтов.

После построения гистограммы выполняется сканирование — операция, которая преобразует гистограмму в префиксную сумму. Сканирование осуществляется с использованием алгоритма Blelloch Scan, который использует разделяемую память на уровне блоков, чтобы ускорить процесс. Используется рекурсивный подход для обработки больших массивов, разделяя их на блоки и объединяя результаты. Сканирование делается в два этапа:

1. На первом этапе вычисляются локальные префиксные суммы внутри блоков.
2. На втором этапе, когда есть несколько блоков, результаты объединяются, добавляя суммы блоков в общие префиксные суммы.

После выполнения сканирования, получаем префиксные суммы, которые позволяют корректно распределить элементы массива в отсортированном порядке. На следующем шаге выполняется сортировка подсчетом, где для каждого элемента массива вычисляется его позиция в отсортированном списке с использованием префиксных сумм гистограммы. Параллельно, с помощью атомарных операций, элементы размещаются в выходном массиве в соответствии с их позицией.

Описание программы

В коде все функции и ядра CUDA реализованы в одном исходном файле.

Основные типы данных:
- int - используется для хранения целых чисел, таких как длина массива и сами элементы массива.
- int* - массивы для хранения данных, например, входной массив чисел, гистограмма, выходной массив.

Основные функции:
- read_input - функция для чтения входных данных. Сначала считывается количество элементов массива, затем сами элементы.
- write_output - функция для записи отсортированных данных в бинарном формате в выходной поток.
- recursive_scan - реализует рекурсивное сканирование данных с использованием алгоритма Blelloch scan. Эта функция выполняет несколько этапов сканирования, сначала вычисляя частичные суммы в блоках, а затем применяя результаты сканирования к глобальным данным.

Реализованные ядра CUDA:

- histogram_kernel - ядро, которое строит гистограмму частоты появления значений в массиве. Для этого используется атомарная операция atomicAdd, которая увеличивает значение в массиве гистограммы по индексу, соответствующему значению из входного массива.
- scan_kernel - ядро для выполнения сканирования (префиксной суммы) на уровне блоков. В каждом блоке используется разделяемая память для хранения промежуточных результатов, а также выполняется последовательное сканирование с использованием алгоритма слияния (reduction). Результаты сканирования блоков сохраняются в массиве.
- add_block_sums_kernel - ядро для добавления сумм блоков к данным гистограммы. Необходимо после рекурсивного сканирования, чтобы корректно рассчитать финальное положение элементов в отсортированном массиве.
- counting_sort_kernel - ядро для выполнения сортировки подсчетом. Оно распределяет элементы по соответствующим индексам в выходной массив на основе значений в гистограмме. Каждый элемент находит свое место, используя атомарную операцию для безопасного обновления индекса в выходном массиве.

Основная функция программы main() - читает входные данные, вызывает CUDA-ядра для выполнения этапов алгоритма сортировки подсчетом и записывает результат в выходной файл.

Результаты

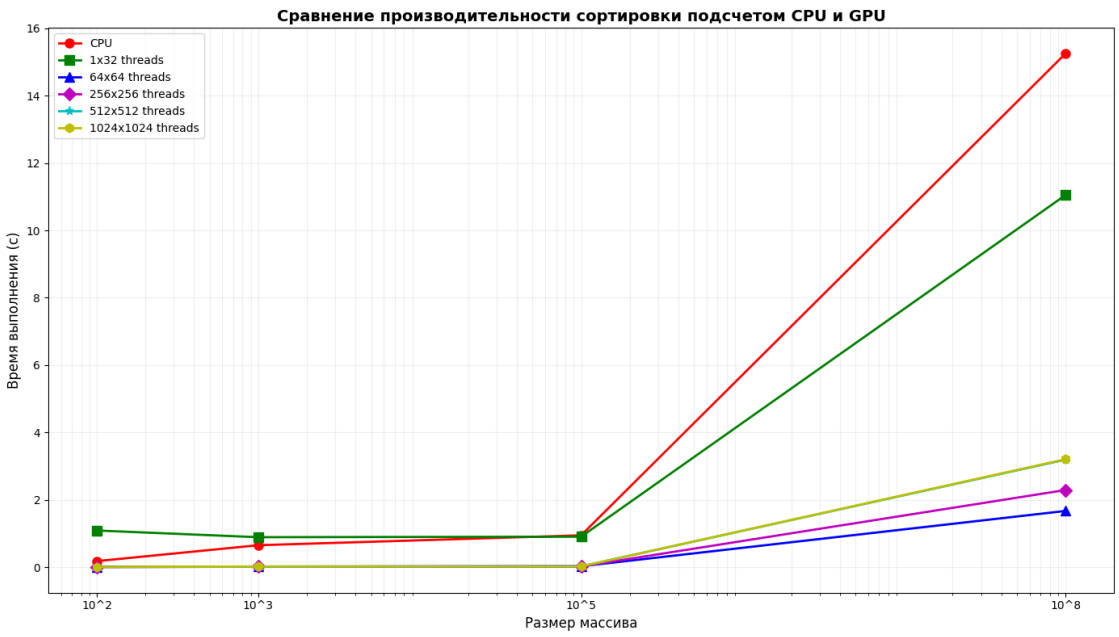
Были проведены тесты производительности GPU реализации сортировки подсчетом с различными конфигурациями запуска ядер для массивов размеров 10^2, 10^3, 10^5 и 10^8 элементов. Замеры времени выполнения проводились 10 раз для каждой конфигурации, и вычислялось среднее значение времени.

Конфигурация	10^2	10^3	10^5	10^8
1 × 32	1092.456 мс	893.891 мс	908.968 мс	11046.844 мс
64 × 64	14.474 мс	22.297 мс	31.858 мс	1673.160 мс
256 × 256	8.264 мс	19.130 мс	25.946 мс	2293.816 мс
512 ×512	8.706 мс	18.131 мс	28.203 мс	3194.881 мс
1024 ×1024	10.379 мс	24.771 мс	27.019 мс	3207.102 мс

Также были выполнены замеры работы с использованием CPU.

Размер массива	Время работы
10^2	183.796 мс
10^3	655.705 мс
10^5	949.618 мс
10^8	15246.659 мс

Ниже представлен график, который иллюстрирует сравнение времени работы GPU с различными конфигурациями и CPU на тестах с размерами массивов 10^2, 10^3, 10^5 и 10^8.



Для профилирования алгоритма был использован массив размером 10^5 и конфигурация блоков 512x512. Ниже представлены результаты из профилировщика nvprof:

```
            Type  Time(%)      Time     Calls       Avg       Min       Max  Name
 GPU activities:   72.09%  3.3275ms         3  1.1092ms  9.5350us  3.3043ms  scan_kernel(int*, int*, int)
                   13.44%  620.27us         2  310.14us  7.8390us  612.44us  add_block_sums_kernel(int*, int*, int)
                    6.17%  284.70us         1  284.70us  284.70us  284.70us  [CUDA memset]
                    3.62%  166.88us         1  166.88us  166.88us  166.88us  counting_sort_kernel(int*, int*, int*, int)
                    3.21%  148.00us         1  148.00us  148.00us  148.00us  histogram_kernel(int*, int*, int)
                    0.77%  35.711us         1  35.711us  35.711us  35.711us  [CUDA memcpy HtoD]
                    0.71%  32.543us         1  32.543us  32.543us  32.543us  [CUDA memcpy DtoH]
      API calls:   96.64%  186.87ms         6  31.145ms  3.5280us  186.76ms  cudaMalloc
                    2.22%  4.2852ms         7  612.17us  10.853us  3.3086ms  cudaDeviceSynchronize
                    0.64%  1.2396ms         6  206.60us  2.8760us  1.0648ms  cudaFree
                    0.21%  399.87us         2  199.93us  107.52us  292.35us  cudaMemcpy
                    0.20%  384.52us         7  54.931us  4.6500us  346.08us  cudaLaunchKernel
                    0.08%  146.05us       114  1.2810us     102ns  59.862us  cuDeviceGetAttribute
                    0.01%  22.798us         1  22.798us  22.798us  22.798us  cudaMemset
                    0.01%  12.559us         1  12.559us  12.559us  12.559us  cuDeviceGetName
                    0.00%  5.0340us         1  5.0340us  5.0340us  5.0340us  cuDeviceGetPCIBusId
                    0.00%  1.7200us         3     573ns     120ns  1.3640us  cuDeviceGetCount
                    0.00%  1.3280us         7     189ns     109ns     551ns  cudaGetLastError
                    0.00%     760ns         2     380ns     158ns     602ns  cuDeviceGet
                    0.00%     646ns         1     646ns     646ns     646ns  cuDeviceTotalMem
                    0.00%     352ns         1     352ns     352ns     352ns  cuModuleGetLoadingMode
                    0.00%     234ns         1     234ns     234ns     234ns  cuDeviceGetUuid
```

Основные вычислительные операции занимают 72.09% от времени выполнения:

- scan_kernel: 72.09% (3.3275 мс)
- add_block_sums_kernel: 13.44% (620.27 мс)
- counting_sort_kernel: 3.62% (166.88 мс)
- histogram_kernel: 3.21% (148.00 мс)

API вызовы:

- cudaMalloc занимает 96.64% времени на управление памятью.
- Операции синхронизации cudaDeviceSynchronize занимают 2.22% времени.

Выводы

В ходе данной лабораторной работы была реализована параллельная версия алгоритма сортировки подсчетом с использованием технологии CUDA для выполнения на GPU.

Сравнение результатов показало, что использование GPU значительно ускоряет процесс сортировки по сравнению с CPU, особенно для массивов размером 10^5 и более,

где ускорение достигало значительных значений. В то же время, для небольших массивов GPU не всегда дает преимущество из-за накладных расходов на запуск и синхронизацию.

Оптимальные результаты были получены при конфигурации блоков 64×64 и 256×256, при которой удалось добиться хорошего баланса между количеством потоков и эффективным использованием параллельных возможностей GPU. Для более крупных массивов конфигурации 512×512 и 1024×1024 продемонстрировали незначительное увеличение времени из-за роста накладных расходов на синхронизацию.

Профилирование с использованием утилиты nvprof показало, что основную часть времени выполнения занимают вычислительные ядра (особенно ядро сканирования), в то время как операции синхронизации и работы с памятью составляют относительно небольшую долю времени. Большая часть времени (96.64%) уходит на операции выделения и освобождения памяти.

Таким образом, данная реализация сортировки подсчетом на GPU продемонстрировала существенные преимущества при обработке больших массивов данных. GPU ускорение является особенно полезным для задач, где необходимо выполнить сортировку больших объемов данных, таких как обработка числовых рядов, анализ данных и задачи машинного обучения.